TAKE-HOME ASSIGNMENT · V2

Orders Console

A small, realistic slice of the kind of system we build at SARH: an operations team works through a large list of orders and registers new ones by hand — and a duplicate order is the worst thing that can happen. Two hard problems, one review, and a few written answers. We care more about how you think than how much you build.

~5 h	4 days	2 + 1	1 repo
Expected effort (Senior adds ~1 h of writing)	Sent Sun 18 Oct · due Thu 22 Oct 2026	Build tasks + one code review	Git link · README · DECISIONS.md

Before you start

This is deliberately small so you can do it well. If you find yourself past six hours, stop, write down what is missing, and submit. An honest, unfinished submission scores higher than a polished one that hides its gaps.

AI assistants and any libraries are allowed. We will ask you to explain, defend and modify your own code live, and DECISIONS.md (section 5) only works if you actually made the decisions.

1. The scenario

An e-commerce operations team receives tens of thousands of orders a month. Agents need to find an order in seconds and register phone orders by hand. Today they do this in a spreadsheet; duplicate orders are their biggest complaint.

Build the smallest full-stack application that solves this properly: a React front end and a Node back end with a real database. Seed it with at least 50,000 orders using a script you write (customer name, email, status, total amount, created date, 1–5 line items).

2. Build tasks (mandatory for everyone)

R1 · A large list that stays fast and stays correct

- Orders list with search (customer name/email), filter (status) and sort (date, amount).
- Must remain smooth with 50,000+ rows: server-side or cursor pagination, and/or virtualised rendering. Never send the full table to the browser. Add the index(es) the query needs and mention them.
- Correctness under load — this is the part we test by hand: **change the filter three times in quick succession. The result of an earlier, slower request must never overwrite the result of the latest one.**

R2 · Create order — no duplicates, ever

- A form to create an order (customer, items, amount). Validate on both client and server.
- The problem: agents double-click, mobile networks retry, and the browser sometimes resends a request after a timeout. None of these may ever produce two orders.
- Solve it however you judge best — but the guarantee has to hold on the server, not only in the UI. Explain your approach and its limits in DECISIONS.md.
- Prove it: a script or test that fires the same create request twice in parallel and shows that exactly one order exists.

3. Code review task (mandatory)

Below is an authentication middleware a previous contractor left in the project. Use it as the starting point for protecting your /api/orders routes (seed at least one admin and one agent user). It contains at least three problems — some security, some design. In your README, list every problem you found and what you changed. Then fix them in your implementation.

```
// auth.ts — "works on my machine"
import jwt from "jsonwebtoken";
const SECRET = "sarh-secret-2026";

export function requireAuth(req, res, next) {
  const token = req.headers.authorization?.split(" ")[1] || req.query.token;
  if (!token) return res.status(401).json({ error: "no token" });
  const payload = jwt.decode(token);              // { sub, role, exp }
  if (!payload) return res.status(401).json({ error: "bad token" });
  req.user = payload;
  next();
}

// routes.ts
router.delete("/orders/:id", requireAuth, async (req, res) => {
  // admin check is done in the React AdminOnly component
  await db.orders.delete({ where: { id: req.params.id } });
  res.json({ ok: true });
});

router.post("/login", async (req, res) => {
  const user = await db.users.findFirst({ where: { email: req.body.email, password: req.body.password } });
  if (!user) return res.status(401).json({ error: "wrong email or password" });
  res.json({ token: jwt.sign({ sub: user.id, role: user.role }, SECRET) });
});
```

Also tell us, in two or three sentences, where you store the token on the client and what attack that choice does and does not protect against.

4. Optional — pick at most one

Only if you are comfortably inside the time budget. Doing neither costs nothing; doing one badly costs more than skipping it.

O1 · The slow stats API

- Expose GET /api/orders/stats (totals by status, revenue today) and make it deliberately slow: 3–6 seconds. The UI must never block, must cancel the request when the user navigates away, and must show a visible error state with one retry.

O2 · Optimistic create

- Show the new order in the list immediately on submit, and roll it back cleanly if the server rejects it. Describe the state you had to keep to make rollback correct.

5. Written answers (mandatory)

DECISIONS.md — three decisions

- For each: the decision, one alternative you seriously considered, and why you rejected it. Five to eight lines per decision. One must be your R2 approach; the other two are yours to choose (state management, pagination strategy, database, folder structure…).
- This is also where you tell us what you used AI tools for. We read this file first.

One product question — answer in your README

- The operations lead asks for a button: “Delete all cancelled orders.” Before you build it, what do you ask her, and what do you propose instead if she cannot answer? Five lines maximum. There is no single right answer; we are looking at whether you ask “why” before “how”.

Senior applicants only — a one-page design memo

- Save it as SCALE.md. Scenario: the table grows to 5 million orders, the team is three developers, and agents in two countries use the console all day. What do you change in your current design, what do you deliberately keep, and what would you refuse to build even if asked? One page; diagrams welcome but not required.
- This replaces the extra build work other assignments ask of seniors. We would rather read one page of judgment than see another 300 lines of code.

6. Technical constraints

Area	Constraint
Front end	React 18+ (Vite or Next.js). TypeScript strongly preferred.
Back end	Node.js — Express, Fastify, NestJS, or Next.js route handlers.
Database	Any real one: SQLite, PostgreSQL, MySQL, MongoDB. In-memory is not acceptable.
Services	No paid or hosted services. Everything runs locally from the README in under five minutes.
Secrets	Commit a .env.example, never a real .env. Seeded passwords may be plain in the README.
AI tools	Allowed and expected. Disclose in DECISIONS.md. The interview is a live walkthrough and modification of your code.

7. What to submit

1. A link to a Git repository (public, or private with access granted to the email below). Keep your commit history — we read it.
2. README: setup, seed and test commands; the code-review findings (section 3); the product question (section 5); and a “what I skipped and why” section. Skipping something and saying so is fine. Skipping something silently is not.
3. DECISIONS.md as described. Senior: SCALE.md as well.
4. A two-minute screen recording or three screenshots showing R1 (including the rapid-filter test) and the R2 duplicate proof.

8. How we evaluate

Each area is scored 0–5 by two reviewers independently. Weights differ by level.

Area	What we look at	Junior	Senior
Correctness	R1 and R2 work as described; the rapid-filter test and the duplicate proof pass; edge cases handled.	25%	20%
Code quality	Structure, naming, separation of concerns, real TypeScript, no dead code.	20%	20%
Performance	Query design, pagination/virtualisation, indexes, request cancellation.	15%	15%
Security review	Section 3: problems found, fixes applied, token-storage reasoning.	15%	15%
Judgment & communication	DECISIONS.md quality, the product question, README honesty, commit history. A clearly stated gap scores higher than a hidden one.	20%	15%
Tests	The R2 proof plus anything else you test.	5%	—
Design memo	Senior only: SCALE.md — specific, grounded in your own code, says what you would refuse.	—	15%

> **What a strong submission looks like**
>
> It runs first time from the README. The list is instant at 50,000 rows and never shows stale results. Firing the create request twice leaves exactly one order. The review finds the real problems, not cosmetic ones. DECISIONS.md reads like a person who weighed two options, not like a changelog.
>
> What it does not need: pixel-perfect design, a component library, full test coverage, Docker, deployment.

9. Rules and logistics

- Individual work. Discussing ideas with others is fine; submitting someone else’s code is not.

- Deadline: Thursday 22 October 2026, end of day (four calendar days from receiving this brief). If you need more time, say so before the deadline — asking is never penalised.
- Questions about the brief: email the address below. If something is ambiguous, make a reasonable assumption and write it down.
- Your code remains yours. We will not use it for any purpose other than this assessment.
- Next step after submission: a 60-minute technical conversation — mostly live changes to your own code.

Submit to

m.alharbi@sarh.ai

Position applied for

Full Stack Developer (React) — ☐ Junior ☐ Senior

Date brief sent / Deadline

18 / 10 / 2026 → 22 / 10 / 2026

Built to endure.